



Volume 5 - Limites, invariants et lecture d'entretien

Ce que le projet garantit, où il s'arrête, et comment justifier les choix

Ce volume rassemble les limites chiffrées, les comportements d'erreur et les points à expliquer lors d'une revue C++/systèmes. Il distingue explicitement les garanties de la maquette des propriétés qui nécessiteraient une architecture de production ou de certification.

Table des matières

1	Table de limites de référence	2
2	Invariants du codec	2
3	Invariants du TCP	2
4	Invariants du pipeline	2
5	STGA : distinction entre erreur de trame et erreur applicative	3
6	Santé : sens exact des seuils	3
7	DSP : ce que l'onde représente réellement	3
7.1	Moyenne glissante	3
7.2	Projection sinus/cosinus	3
8	Port discovery : limite de sécurité et de sémantique	4
9	Fichiers STGF	4
10	I/O et erreurs disque	4
11	Questions probables en entretien et réponses techniques	4
11.1	Pourquoi un CRC et pas un hash cryptographique?	4
11.2	Pourquoi UDP et TCP?	4
11.3	Pourquoi plusieurs décodeurs mais un writer?	4
11.4	Pourquoi une file bornée?	4
11.5	Pourquoi le port-check ne vérifie-t-il pas l'identité STGS?	4
11.6	Pourquoi les valeurs de santé ne sont-elles pas des seuils métier?	5
12	Ce qu'il faudrait ajouter pour une production critique	5

1 Table de limites de référence

Élément	Limite	Origine / conséquence
Port utilisateur	1..65535	Type u16, port 0 refusé par la CLI.
Plage diagnostic	256 ports max	Empêche un scan local non borné.
Timeout probe	1..60000 ms	Garde-fou CLI.
Header STGS	23 octets	Layout v1 verrouillé par <code>static_assert</code> .
CRC STGS	4 octets	CRC-32/ISO-HDLC.
Payload STGS	4096 octets max	Limite applicative volontaire.
Trame STGS	27..4123 octets	Header + payload + CRC.
Message STGA	4086 octets max	4096 - 10 octets de header texte.
Signal STGA	1018 samples max	4096 - 22, puis 4 octets/sample.
Sample rate	1..65535 Hz	u16 STGA.
Fréquence signal	$0 < f < f_s/2$	Nyquist strict.
Décodeurs station	1..256	Garde-fou CLI.
Capacité de file	1..1 000 000	Production toujours bornée.
Filtre moyenne	fenêtre impaire 1..1017	Centré et borné au bloc maximal.
Frames simulateur	1..10 000 000	Garde-fou génération.

2 Invariants du codec

`decodeFrame()` vérifie dans cet ordre : taille minimale, magic, version, `PAYLOAD_LEN`, taille totale, CRC, température finie, batterie, statut. Cet ordre évite de lire des champs hors limites et évite de construire une `TelemetryFrame` avant validation d'intégrité.

Décision de validation

```
bytes → size >= 27 → magic == STGS → version == 1 → payload <= 4096 → size
exacte → CRC correct → temperature finie → battery <= 100 → status connu →
TelemetryFrame
```

Les corruptions attendues sont des valeurs `FrameError`, pas des exceptions. Cela évite d'utiliser les exceptions comme contrôle de flux normal sur une entrée bruitée.

3 Invariants du TCP

- un `StreamFrameExtractor` par client ;
- les frontières de `recv()` ne sont jamais considérées comme des frontières de trame ;
- la liste `pollfd` est un snapshot cohérent du nombre de clients avant `poll()` ;
- les clients sont traités en ordre inverse pour permettre `erase()` sans décaler les indices restant à traiter ;
- les données sont drainées même quand `POLLHUP` accompagne les derniers octets ;
- les nouveaux clients ne sont acceptés qu'après traitement du snapshot courant.

Source : `src/NetworkServerTcp.cpp`, lignes 105–246

4 Invariants du pipeline

- la séquence est attribuée avant le décodage parallèle ;

- un worker ne modifie ni santé ni fichier ;
- un seul writer applique les effets de bord ordonnés ;
- une séquence manquante à la fermeture déclenche une erreur ;
- la file brute est fermée et drainée avant la file décodée ;
- la première exception de worker est capturée et relancée sur le thread principal.

5 STGA : distinction entre erreur de trame et erreur applicative

Une trame peut avoir un CRC correct mais contenir une enveloppe STGA malformée. Dans ce cas :

1. `decodeFrame()` accepte la trame ;
2. le compteur `decoded` augmente ;
3. `decodeApplicationPayload()` retourne une erreur applicative ;
4. `applicationErrors` augmente et un warning est loggé ;
5. la trame externe reste exportée, car son intégrité STGS est valide.

Ce découplage permet de faire évoluer la couche applicative sans confondre intégrité transport et interprétation métier.

6 Santé : sens exact des seuils

La fenêtre contient à la fois trames valides et rejets. Un rejet compte dans le taux de rejet mais jamais comme télémétrie critique. Une trame valide CRITICAL ou SAFE_MODE compte comme critique.

Valeurs par défaut

Fenêtre 100, minimum 20, dégradation à 10% de rejets ou 8 critiques, récupération à 3% de rejets et 3 critiques maximum. Ce sont des paramètres de démonstration, pas une norme spatiale.

7 DSP : ce que l'onde représente réellement

Le mode signal ne simule pas une onde RF analogique. Il transporte un bloc d'échantillons numériques représentant un sinus synthétique et éventuellement un bruit gaussien. Le filtre `sine-projection` suppose connue la fréquence nominale incluse dans les métadonnées STGA.

7.1 Moyenne glissante

Pour chaque point, la fenêtre impaire est centrée et tronquée aux bords. Complexité de l'implémentation actuelle : approximativement $O(NW)$, acceptable pour les petits blocs de démonstration mais optimisable par somme glissante pour de très grands flux.

7.2 Projection sinus/cosinus

Les coefficients sont estimés par :

$$a = \frac{2}{N} \sum x[n] \sin(\omega n), \quad b = \frac{2}{N} \sum x[n] \cos(\omega n)$$

et la reconstruction par $dc + a \sin(\omega n) + b \cos(\omega n)$. L'amplitude estimée vaut $\sqrt{a^2 + b^2}$.

Limite de la métrique SNR

Le bruit est défini comme **raw-filtered**. Le SNR est donc dépendant du filtre et constitue une métrique de démonstration, pas une mesure RF instrumentale.

8 Port discovery : limite de sécurité et de sémantique

Les fonctions de probe acceptent uniquement le loopback IPv4. Elles sont séquentielles, limitées à 256 ports et utilisent un timeout borné. Cette décision réduit volontairement la fonctionnalité : l'objectif est de diagnostiquer plusieurs processus locaux STGS, pas de fournir un scanner réseau.

9 Fichiers STGF

Le fichier commence par **STGF** + version u32 big-endian. Chaque entrée contient longueur u32 + trame STGS complète. Avant allocation, la longueur doit être comprise entre 27 et 4123. Le replay ne contourne pas le codec : les trames repassent par le même `decodeFrame()` que le réseau.

10 I/O et erreurs disque

`TelemetryOutput` et `FrameFileWriter` vérifient l'état du flux après écriture et au flush explicite. Une erreur disque remonte donc comme exception, est capturée par la stratégie d'arrêt du pipeline si elle survient dans le writer, puis finit par faire échouer le processus plutôt que de produire silencieusement un fichier présenté comme complet.

11 Questions probables en entretien et réponses techniques

11.1 Pourquoi un CRC et pas un hash cryptographique ?

Le CRC vise la détection d'erreurs accidentelles sur la trame de démonstration, pas l'authenticité. Pour une menace active, il faudrait ajouter un mécanisme cryptographique (par exemple MAC/AEAD) et une gestion de clés ; ce n'est pas le rôle de CRC-32.

11.2 Pourquoi UDP et TCP ?

UDP permet un message par datagramme et une architecture simple ; TCP démontre le problème réel de framing sur flux, fragmentation, multi-clients et backpressure. Le codec reste commun.

11.3 Pourquoi plusieurs décodeurs mais un writer ?

Le décodage/CRC est parallélisable ; l'ordre des effets de bord doit rester déterministe. Le writer unique simplifie l'invariant d'ordre et évite un verrou complexe autour du fichier et du moniteur de santé.

11.4 Pourquoi une file bornée ?

Sans borne, un producteur plus rapide que les consommateurs transforme la surcharge en croissance mémoire. La backpressure déplace le problème vers le débit, observable et maîtrisable.

11.5 Pourquoi le port-check ne vérifie-t-il pas l'identité STGS ?

Le protocole actuel est unidirectionnel et n'a pas de handshake. Ajouter une identité exigerait de concevoir une négociation/commande bidirectionnelle distincte. Le code préfère afficher honnêtement OPEN = listener TCP.

11.6 Pourquoi les valeurs de santé ne sont-elles pas des seuils métier ?

Parce que le projet est une vitrine technique. Les seuils sont configurables et explicitement documentés comme valeurs de démonstration pour éviter de leur donner une autorité qu'ils n'ont pas.

12 Ce qu'il faudrait ajouter pour une production critique

Selon le contexte réel : authentification/chiffrement, configuration mission externalisée, observabilité structurée, stratégie HA, tests de charge bout-en-bout, fuzzing continu, limites de connexions, gestion de quotas, compatibilité/versioning de protocole plus formalisés, métriques exportables, packaging/installation et éventuellement exigences de temps réel/certification.

Pourquoi documenter explicitement les limites ?

Une documentation technique crédible ne prétend pas que la maquette garantit ce qu'elle n'implémente pas. Savoir définir la frontière de garantie est souvent plus important en revue d'architecture que d'afficher une longue liste de fonctionnalités.